Ambiguous Self-Comparison Predicates

#sql-server #correctness #scoping #refactoring

What the Pattern Looks Like

A predicate inside a subquery references a column name that exists in both the inner and outer scope, without qualifying which one it means. SQL Server's name resolution always prefers the innermost scope, so both sides of the comparison resolve to the same column. The predicate becomes a self-comparison, which is true for every non-NULL row, and the intended cross-scope filter is silently lost.

The canonical example, taken from a production stored procedure:

```
Declare @RestrictAddrBank Table (AddrBank varchar(2))

Insert Into @RestrictAddrBank Values ('BR'), ('BL')

Select OrderStatus
From OeOrder
Where Exists (
    Select AddrBank
    From @RestrictAddrBank
    Where AddrBank = AddrBank
)
```

The author's intent is "include OeOrder rows whose AddrBank matches one of the values in @RestrictAddrBank." What actually executes is "include OeOrder rows whenever @RestrictAddrBank has at least one non-NULL row." Both sides of the inner `AddrBank = AddrBank` resolve to the inner table's column, so the predicate is `R.AddrBank = R.AddrBank`, which is true for every non-NULL row. The intended outer reference (`OeOrder.AddrBank`) is silently dropped.

The query runs without error, returns rows, and looks reasonable in code review. The bug is invisible until someone notices that the filter parameter has no observable effect.

Why It Hurts

This is a correctness bug, not a performance bug. The fix changes the query's result set, which changes the proc's contract with its callers.

Three concrete consequences:

- **The filter parameter has no effect.** Every value of `@AddrBankFilter` produces the same result set, as long as the filter table has at least one row. Callers that supplied a narrow filter were getting fleet-wide totals, and never noticed because the absolute counts were plausible.
- **Performance signals are misleading.** The proc's read counts are larger than they should be, because the optimizer is materializing the unfiltered set. Triage based on read counts will identify the proc as expensive but will not identify the cause as a missing filter, since the SQL text "looks like" it filters.
- **Fixing the bug changes downstream behavior.** Once the filter is applied correctly, callers passing `@AddrBankFilter = 'BR'` will see a strict subset of rows compared to the broken version. Any consumer that was implicitly relying on the unfiltered totals will see different numbers.

How to Recognize It

Three signals to look for during a code review:

- **A subquery references a column name without an alias prefix.** Subqueries on tables that share column names with their outer query are the highest-risk site for this bug. `WHERE AddrBank = AddrBank` should never appear in any code review without prompting a question about which scope each side resolves to.
- **A filter parameter that the proc claims to honor, but the cost numbers and result shapes do not show evidence of.** If `@AddrBankFilter = 'BR'` and `@AddrBankFilter = 'BL'` produce identical read counts and identical durations on a busy production system, the filter is probably not being applied.
- **`Exists` or `In` subqueries that reference the inner table's column on both sides of an equality comparison.** This is the canonical shape. The intent is almost always cross-scope; the implementation is almost always intra-scope.

The Fix

Always alias both tables and qualify every column reference. The fix is mechanical:

```
-- Broken
Where Exists (
    Select AddrBank
    From @RestrictAddrBank
    Where AddrBank = AddrBank
)

-- Fixed
Where Exists (
    Select 1
    From @RestrictAddrBank As R
    Where R.AddrBank = O.AddrBank
)
```

Two changes. First, alias the inner table (`R`) and the outer table (`O`, declared at the FROM clause of the outer query). Second, qualify both sides of the predicate (`R.AddrBank = O.AddrBank`). The intent is now unambiguous and the engine cannot silently re-resolve either side.

The `Select 1` form is also worth adopting. Inside an `Exists` subquery, the projected column list is irrelevant. `Select 1` makes that explicit and removes a column reference that could be misread as part of the filter.

When the Bug Lurks Hardest

Three scopes amplify the risk:

- **A column appears in both the outer and inner table by the same name.** Lookup tables on the same domain (`AddrBank`, `Status`, `OrderId`, `GroupNum`) are the canonical examples. The chance of accidentally writing `Where Col = Col` is highest when the column appears in both tables.
- **The author copies the inner column into the SELECT list of the subquery and then writes a WHERE clause from memory.** The unqualified name in the SELECT list primes the unqualified name in the WHERE clause, and both resolve to the inner table.
- **The proc has been in production for a long time.** If the bug has been there since the proc was written, no one noticed it broke the filter, because no one ever ran a comparison against a version where the filter was actually applied.

Detection in Practice

A few systematic ways to catch this class of bug:

- **Search for `Where <col> = <col>` shapes** in the code base, especially in subqueries. A regex over stored procedure source files for `Where\s+\w+\s*=\s*\w+\b` (with the same identifier on both sides) finds candidates fast.
- **Read every subquery looking for unqualified column names.** Any unqualified column name in a subquery that references a table that shares the column with the outer scope is a candidate for this bug.
- **Run the proc with a deliberately narrow parameter and a deliberately broad parameter,** and verify the row counts differ. Identical counts across "filtered" and "unfiltered" calls is a smoking gun.

Related Concepts

- Correlated Subqueries to CTEs: a related class of bug, where the outer-scope reference is intended but not optimal. The two patterns share a root cause: subqueries that depend on outer scope are easy to mis-read.
- Non-SARGable Predicates: not the same problem, but the same lesson applies. Predicates need to be inspected carefully, because the optimizer's plan will execute exactly what was written, not what was meant.